

AtlasForge — AI Intelligence Graph

Architecture Document · v0.1.0

System Overview

AtlasForge is a scalable, fault-tolerant data intelligence pipeline that collects, processes, and normalizes data about AI startups, products, research papers, news, and jobs. The current implementation runs locally on a single machine using SQLite, FastAPI, and React. Every component is designed so that swapping in PostgreSQL, Kafka, and Redis later requires zero application-code changes — only an environment variable and infrastructure provisioning.

Current Architecture (5 Layers)

Layer	Components
Presentation	React 18 + TypeScript + Vite · Tailwind + Lucide + Recharts · 10 routed pages (Command Center, Papers, Sta
API	FastAPI + uvicorn · Pydantic v2 · CORS to :5173 · Endpoints: /health, /dashboard, /{module}, /pipeline/runs, /in
Ingestion	6 ingestors: arXiv (Atom XML), GitHub enrichment (REST), RSS news (4 feeds), Jobs (RemoteOK + Arbeitnow
LLM Tier	Multi-tier adapter: Gemini 1.5 Flash → Groq Llama 3.1 → DeepSeek. Exponential backoff + jitter (4 attempts, 0
Storage	SQLite via aiosqlite. 7 tables: papers, startups, products, news, jobs, entities, entity_aliases, pipeline_runs. Uni

Data Flow

Ingestor → async fetch → normalize → unique-key dedup → optional LLM extraction (with deterministic fallback) → DB write + PipelineRun audit row. Frontend polls /dashboard + /pipeline/runs every 30s. All ingestors are idempotent and re-runnable.

Verified End-to-End (no fabrication)

30 real arXiv papers ingested from cs.AI/LG/CL/CV/RO/IR + stat.ML. 3 papers enriched with real GitHub stars (e.g. 320★). 69 RSS items fetched, 7 kept within 24h freshness, 62 rejected for staleness. 13 entity clusters resolved from real news mentions. 5/5 test suites pass (LLM helpers, LLM adapter, entity resolver, export, smoke).

Data Integrity & Honesty

Hard Constraints

Zero budget: only free, open-source tools. SQLite, aiohttp, feedparser, RapidFuzz, ReportLab, free tiers of Gemini/Groq/DeepSeek/GitHub. No paid APIs, hosting, or databases.

No hallucinations: every record comes from a legitimate source or is labeled 'deterministic'. LLM-extracted values carry method + provider + confidence metadata so downstream consumers can distinguish.

Honest metrics: the dashboard distinguishes between real collected records, failed/pending records (reported separately in the pipeline log), and architectural projections (clearly labeled). The UI shows 'No Records Yet' or 'Backend Offline' rather than fabricating numbers.

Reliability & Error Handling

Idempotent ingestors: every ingestor uses (source, source_id) unique constraints so re-running cannot create duplicates — only updates existing rows.

Per-host rate limiter: a token-bucket implemented in `core/http.py` enforces a minimum 2s gap between requests to the same host, with global override via `ATLAS_RATE_LIMIT` env var. Polite to public APIs.

Per-provider backoff: 429 and 5xx responses from LLM providers trigger exponential backoff with 25% jitter, max 4 attempts before moving to next tier. 413 payloads are pre-chunked (12k chars + overlap) and results merged as JSON arrays.

Pipeline audit: every run writes a PipelineRun row with run_id, module, source, status, fetched/new/updated/failed counts and a free-text message. Visible at `/pipeline` in the UI.

Schema & Migration Posture

Decision	Rationale
SQLAlchemy 2 typed maps	Same model files work with SQLite + Postgres
JSON columns for arrays	authors_json, categories, extracted — Postgres JSONB equivalent
UniqueConstraint on (source, source_id)	Database-level, not app-level
Timestamp with timezone	All datetimes stored UTC; no naive datetime bugs
Async everywhere	aiosqlite → asyncpg swap is a one-env-var change
No ORM migrations yet	For demo: <code>Base.metadata.create_all</code> on startup

Observability (current)

Structured stdout logging with timestamps and module names. **Pipeline log** endpoint exposes every run with full audit. **Health endpoint** reports service version + UTC time. Production roadmap (page 3) adds OpenTelemetry traces + Prometheus metrics.

500k-Record Production Roadmap

Architecture Document · Page 3 of 3

Capacity Targets

	Current (Local)	Production Target
	~50 (verified)	500,000
	≤100	≥10,000
	≤10	10,000 sustained
	n/a (single user)	≤300ms
	n/a	99.9% (≤8.7h/yr downtime)
	none	30-day PITR + geo-redundant

Target Architecture (6 Tiers)

Tier	Components
Edge / CDN	Cloudflare or Fastly · TLS + WAF · Brotli + HTTP/3 · Edge rate limit (10k req/min/IP)
Stateless API	FastAPI on GKE/EKS · HPA 3–50 pods · OIDC + per-tenant JWT · OpenTelemetry → Jaeger · Cooperative 42
Streaming Bus	Kafka 3 brokers · topics: raw.arxiv, raw.news, raw.jobs, raw.papers.github, normalized.entities · exactly-once s
Worker Tier	Ingestor workers (Celery/RQ) 5–30 concurrent · LLM workers (reserved TPM + circuit breaker) · Entity resolver
Cache + Search	Redis Cluster (hot reads, dedup) · OpenSearch 3 shards (full-text, facets) · Qdrant (vector store for semantic d
Storage	PostgreSQL 16 primary + 2 replicas · pgvector + JSONB GIN · Sharded by source_id hash · S3/GCS cold stor

Cost Projection (clearly labeled)

This is an architectural projection, not actual spend. Based on public cloud pricing as of late 2025. Actual costs depend on egress, storage growth, and LLM token usage.

	Spec	Monthly (USD)
)	db.r7g.2xlarge, 500GB, 2 replicas	\$420
	3 brokers, m5.large, 1TB	\$390
ers)	8 vCPU avg, autoscaling	\$280
	3 nodes, cache.t3.medium	\$95
	3 data + 3 master, t3.medium	\$310
ers	100M requests/mo	\$25
	Gemini + Groq + DeepSeek free + buffer	\$60
	Honeycomb + Grafana Cloud free tiers	\$0

Migration Steps (incremental)

1. **Storage swap:** set `ATLAS_DB_URL=postgresql+asyncpg://...` and re-run. No code changes — SQLAlchemy maps the existing models.
2. **Async worker offload:** add a Celery/RQ layer. Move ingestors into tasks. API endpoints switch from *await ingestor.run()* to *task.delay()*.
3. **Kafka between API and workers:** topics become the durability boundary. API only publishes; workers consume + dedup.
4. **Read replicas + Redis:** add pgBouncer in front, Redis for hot-path dashboard counters, OpenSearch for full-text.
5. **Observability:** instrument with OpenTelemetry, export to Jaeger/Honeycomb. Add Prometheus + Grafana for SLO dashboards.

Risks & Mitigations

Free-tier LLM rate limits — mitigated by multi-tier fallback and respectful per-provider budgeting.

Public RSS feed reliability — mitigated by per-feed failure isolation; 2 of 4 feeds failed during the demo run and were honestly logged.

Entity resolution correctness — mitigated by always returning *method* + *confidence* + *reasoning* so downstream consumers can reject low-confidence matches.

Schema drift during migration — mitigated by SQLAlchemy typed maps + JSON columns for arrays, validated by the existing test suite.